

**AI Enabled Accident Detection and Alert System Using IoT and
Deep Learning for Smart Cities**

by

23BEC1030	S S V MRIDULA
23BEC1048	ARUSHI PETHKAR
23BEC1084	NANDINI SHARMA

A project report submitted to

Dr.V. PRAKASH

SCHOOL OF ELECTRONICS ENGINEERING

in partial fulfilment of the requirements for the course of

BECE403E – EMBEDDED SYSTEM DESIGN

in

**B. Tech. ELECTRONICS AND COMMUNICATION
ENGINEERING**



Vandalur – Kelambakkam Road

Chennai – 600127

NOVEMBER 2025

BONAFIDE CERTIFICATE

Certified that this project report entitled “**AI Enabled Accident Detection and Alert System Using IoT and Deep Learning for Smart Cities**” is a Bonafide work of **S S V Mridula – 23BEC1030, Arushi Pethkar – 23BEC1048 and Nandini Sharma – 23BEC1084** who carried out the Project work under my supervision and guidance for **BECE403E-EMBEDDED SYSTEM DESIGN**.

Dr. V. PRAKASH

Associate Professor

School of Electronics Engineering (SENSE),

VIT University, Chennai

Chennai – 600 127.

ABSTRACT

This project presents an AI-enabled accident detection and alert system designed for smart city infrastructure, inspired by the work of Pathik *et al.* [1]. The system integrates Internet of Things (IoT) devices and deep learning algorithms to detect road accidents in real time and automatically alert nearby emergency services. The proposed model combines multiple sensory inputs such as force sensors, IMU modules, and camera-based visual detection to improve the accuracy of detection and minimize false alarms.

By leveraging GPS and GSM modules, the system efficiently transmits accident coordinates to the concerned authorities, thereby reducing the time lag between accident occurrence and emergency response. The deep learning model trained on real-time traffic and accident data enables classification of accident severity and supports predictive analytics for identifying accident-prone zones. The system's modular embedded architecture ensures scalability, low power consumption, and adaptability across diverse environments. This design demonstrates a synergistic relationship between embedded hardware and cloud-based AI analytics, offering a robust, intelligent, and sustainable solution for modern urban safety management.

TABLE OF CONTENTS

S. NO.	TITLE	PAGE NO.
	ABSTRACT	
1	INTRODUCTION	
	1.1 MOTIVATION	
	1.2 OBJECTIVES	
	1.3 APPLICATIONS	
2	EMBEDDED SYSTEM DESIGN PROCESS	
	2.1 REQUIREMENTS	
	2.2 SPECIFICATION	
	2.3 ARCHITECTURE: HARDWARE AND SOFTWARE	
	2.4 COMPONENTS: HARDWARE & SOFTWARE	
	2.5 SYSTEM INTEGRATION	
3	SYSTEM IMPLEMENTATION	
	3.1 SOFTWARE –CODING AND ANALYSIS (SNAPSHOTS OF CODING AND SIMULATION ENVIRONMENT)	
	3.2 HARDWARE – CONNECTION DIAGRAM/SCHEMATIC, PCB DESIGN SCREENSHOTS, PHOTOS OF HARDWARE IMPLEMENTATION	
	3.3 RESULTS & DISCUSSION	
	3.4 COST ANALYSIS	
4	CONCLUSION AND FUTURE WORK	
	4.1 CONCLUSION	
	4.2 FUTURE WORK	
5	REFERENCES	

1. INTRODUCTION

1.1 MOTIVATION

Accidents and corresponding fatalities remain as a big concern worldwide, motivating us to develop an efficient and intelligent system capable of monitoring and detecting accidents in real time to prevent undesirable circumstances and reduce response delays. This work aligns with the vision of smart cities, aiming to enhance safety through advanced technologies. Our approach integrates IoT, cloud computing, deep learning models, and various hardware modules, microprocessors, and communication protocols - key components under the umbrella of Embedded Systems Design; resulting in a clear, purpose-driven system that contributes to intelligent and responsive urban infrastructure.

1.2 OBJECTIVES

1. To detect accidents using the integration of IoT and deep learning networks.
2. To minimize the false detection rates by changing the conventional way of implementation. A system that is not inside the vehicles but monitors them with a new perspective, with a greater coverage range.
3. To send the alert notifications to the nearby services using the GPS module and the shortest path distance formula.
4. To predict the peak accident zones and implement preventive measures there.
5. To make the procedure sustainable. We can effectively use solar power supply in the later stages.

1.3 APPLICATIONS

The proposed AI-enabled accident detection and alert system have several real-world applications in the domain of smart transportation and public safety. It can be deployed in smart city road safety networks to monitor traffic and detect accidents in real time, enabling instant alerts to emergency services for quicker response and rescue. The system can also form part of a real-time accident alert network, where data from multiple units are sent to control centres to coordinate medical and traffic management efforts. Furthermore, it enables intelligent transportation systems by collecting and analysing data to identify accident-prone zones and patterns, allowing authorities to implement preventive measures. Through continuous monitoring and cloud-based analytics, the system contributes to data-driven decision-making that enhances urban safety, reduces response time, and helps build more resilient and efficient road infrastructure.

2. EMBEDDED SYSTEM DESIGN PROCESS

2.1 REQUIREMENTS

The proposed system aims to design and implement an AI-enabled accident detection and alert mechanism that automatically senses, verifies, and analyzes vehicular accidents in real time using a combination of IoT sensors and deep learning techniques. The system continuously collects data from force and IMU (Inertial Measurement Unit) sensors to monitor impact intensity and motion parameters. When these values exceed a predefined threshold, a potential accident is detected. An alarm is triggered, initiating a 30-second response window during which the passenger can press a false alarm button to cancel the alert in case of a minor event.

If no cancellation occurs within this period, the system confirms the accident and retrieves the GPS coordinates of the location. These coordinates are then transmitted to the microcontroller unit (MCU), which uses the GSM module to send an emergency message containing the accident location to the registered contact or nearby emergency services. Simultaneously, a Raspberry Pi module activates an attached camera, captures real-time images of the accident scene, and uploads them to the cloud server. On the cloud, a deep learning model analyzes the captured images to assess the degree and severity of the accident, helping authorities prioritize emergency response.

2.2 SPECIFICATIONS

1. **Continuous Data Monitoring:** The Arduino microcontroller continuously collects sensor data from the force sensor and IMU sensor (MPU9250/55) to monitor acceleration, orientation, and collision intensity of the vehicle.
2. **Threshold Detection and Event Triggering:** When either of the sensor readings surpasses the predefined impact threshold, the Arduino detects a potential accident and triggers an emergency alert sequence.
3. **False Alarm Handling:** Once triggered, a 30-second countdown begins. During this interval, if the passenger presses the false alarm push button, the system cancels the alert and resumes monitoring. This prevents unnecessary emergency notifications during minor shocks or abrupt braking.
4. **Accident Confirmation and Location Acquisition:** If the false alarm button is not pressed within the specified time, the accident is confirmed. The Arduino retrieves the vehicle's GPS coordinates from the NEO-6M GPS module.
5. **Data Transmission to Raspberry Pi:** The Arduino sends the detected event information and GPS data to the Raspberry Pi via serial communication. The Pi acts as the central processing and cloud interface unit.
6. **Image Capture and Cloud Upload:** Upon receiving the accident signal, the Raspberry Pi activates the Pi Camera to capture images of the accident scene. These images, along with the GPS location, are uploaded to a cloud server for storage and further analysis.
7. **AI-Based Accident Severity Analysis:** A deep learning model deployed on the cloud analyses the uploaded images to evaluate the degree of damage or severity of the accident. This automated assessment aids emergency responders in prioritizing their actions.
8. **Emergency Notification via SMS:** Based on the confirmed event, the Raspberry Pi uses a Python-based GSM interface (or web-based SMS API) to send an automated emergency message containing the accident location and severity to pre-registered emergency contacts or authorities.

2.3 ARCHITECTURE: HARDWARE & SOFTWARE

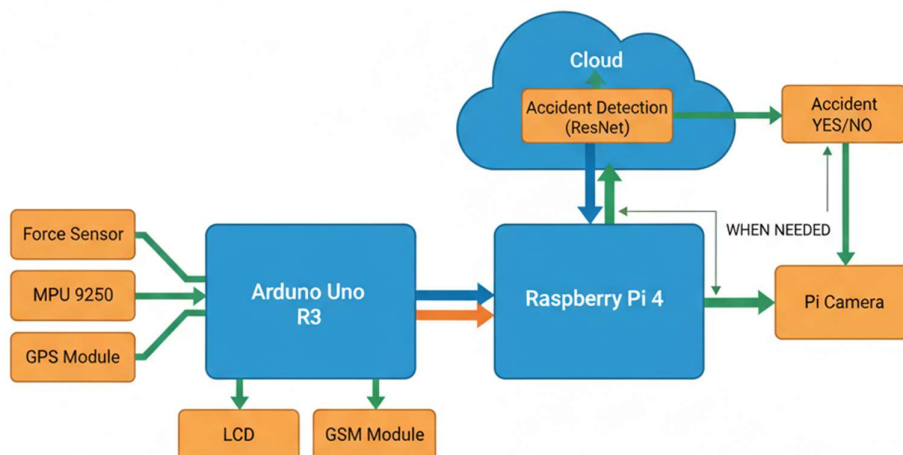


Figure 1: Hardware Block Diagram

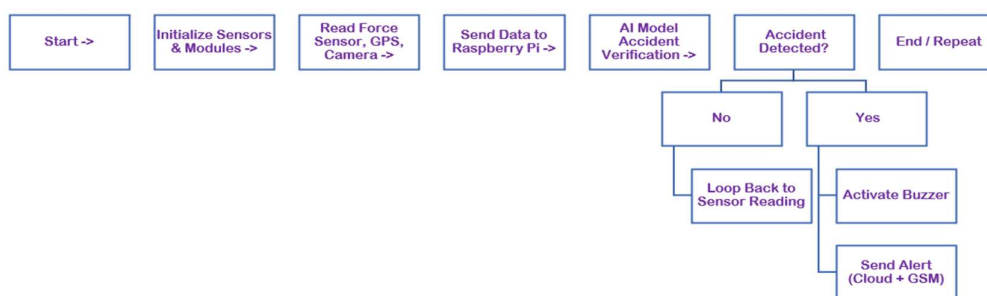


Figure 2: Software Flow Diagram

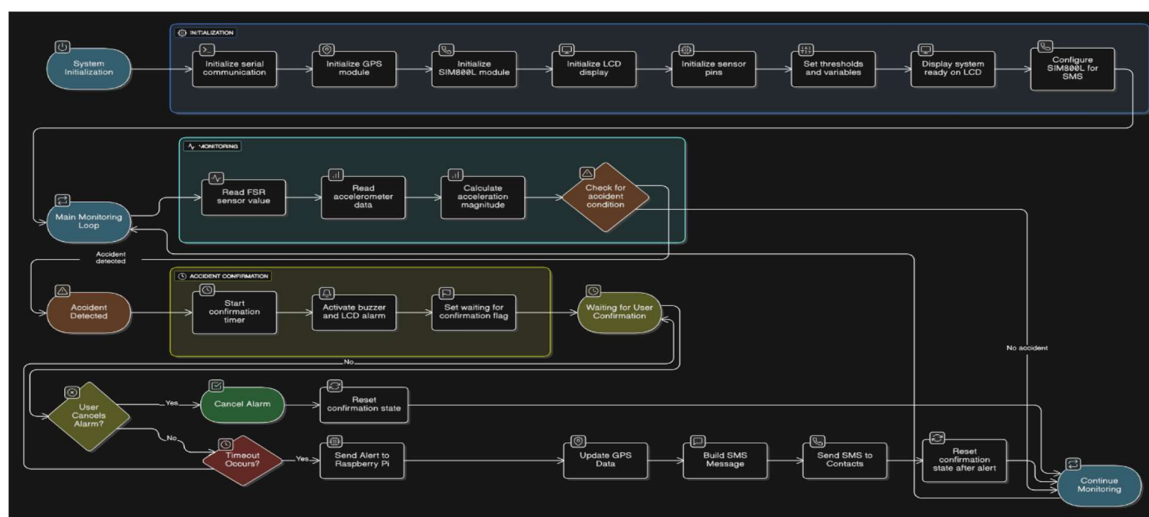


Figure 3: Software Logic Diagram

2.4 COMPONENTS: HARDWARE & SOFTWARE

Hardware Used: -

Component	Model/Variant	Function	Justification
Force Sensor	-	Detect impact force	Adjustable, accurate sensing
MCU 1	Arduino Uno	Reads sensor, triggers alarm	Simple, reliable detection
Wifi Module	NodeMCU ESP8266	Coordinates system, sends data	Wi-Fi-enabled master control
MCU 2	Raspberry Pi 3B+	Image capture & DL analysis	Handles media + cloud upload
Camera	Pi Camera	Accident scene capture	Compact, Pi-compatible
IMU Sensor	MPU 9250/55	Detect acceleration, and inertia	Detects accidents - I/P data
GSM	SIM800L GPRS	SMS alerts	Works without internet
GPS	NEO-6M	Accident location	Reliable tracking
Display	16×2 LCD	Show GPS location	Simple visual feedback
Power	-	5 V DC + 4V DC for GSM	Stable power supply

Software Tools Used: -

Tool	Purpose	Justification
Arduino IDE	Program ATmega328 & NodeMCU	Open-source, supports both MCUs
Python (with OpenCV)	Pi image/video capture & processing	Standard for camera and CV tasks
TensorFlow (Deep Learning)	Accident severity analysis	Widely used ML/DL framework
Google Colab	Cloud training of DL models	Free GPU, easy collaboration
MQTT	IoT communication	Lightweight, reliable protocol
Raspbian OS	OS for Raspberry Pi	Optimized Linux for Pi hardware
Twilio API	integrated with Raspberry Pi	Cloud-based SMS alerts without GSM hardware dependency

2.5 SYSTEM INTEGRATION

The architecture of the proposed system is divided into two main layers: an Arduino-based IoT module and a Raspberry Pi-based processing module. The Arduino handles data collection from the force sensor and IMU sensor using the I²C protocol, which allows multiple devices like sensors and the 16×2 LCD display to communicate efficiently over just two lines (SDA and SCL). When sensor readings exceed a preset threshold, the Arduino sends an accident alert along with GPS coordinates to the Raspberry Pi through serial UART communication. The Raspberry Pi then activates the Pi Camera to capture images of the accident scene and uploads them to the cloud, where a deep learning model analyses the severity of the accident. It also manages SMS alerts via a GSM interface or online API to notify emergency contacts. This integrated hardware - software architecture ensures smooth data flow, accurate detection, and intelligent response for real-time accident monitoring.

3. SYSTEM IMPLEMENTATION

3.1 SOFTWARE (Coding and Analysis)

The software implementation was divided into two main parts: the IoT module (Arduino) and the Processing module (Raspberry Pi).

In the IoT module, coding was done using the Arduino IDE with libraries such as Wire.h, SoftwareSerial.h, TinyGPS++.h, and LiquidCrystal_I2C.h. The program continuously reads sensor data from the Force Sensor (FSR) and IMU Sensor (MPU6050) through the I²C protocol. When the impact or acceleration values exceed predefined thresholds (FSR > 600 or acceleration > 3g), the system identifies a potential accident and triggers an alarm. A 30 - second timer is then activated, allowing the passenger to cancel the alert using a push button if it is a false alarm. If no input is received, the accident is confirmed and the Arduino retrieves GPS coordinates via the NEO-6M module. It then activates the ALERT pin, sending a signal to the Raspberry Pi for further processing. The LCD display (connected via I²C) shows system status such as “System Ready,” “Alarm for Accident,” and “Accident Confirmed.” The SIM800L GSM module is initialized to send emergency messages, and the system uses serial UART communication to transmit accident data and coordinates to the Raspberry Pi.

On the Raspberry Pi side, a Python script handles serial data reception, camera control, and cloud operations. When the accident signal is received, the Pi Camera captures images of the scene and uploads them to the cloud for AI-based severity analysis using OpenCV and TensorFlow. The Pi also sends an SMS alert (through GSM or an online API) containing the GPS coordinates and severity level to registered emergency contacts. The analysis verified that both parts of the system worked synchronously - ensuring reliable detection, accurate location tracking, and automated alert transmission.

Code for IoT (Arduino):

```
#include <Wire.h>

#include <SoftwareSerial.h>

#include <TinyGPS++.h>

#include <LiquidCrystal_I2C.h>

//Pins

const int FSR_PIN = A0;

const int BUZZER_PIN = 8;

const int CONFIRM_PIN = 7;

const int ALERT_PIN = 9;

//Thresholds

const int FSR_THRESHOLD = 600;

const float ACC_THRESHOLD = 3.0;
```

```
//MPU6050

const byte MPU_ADDR = 0x68;

//GPS setup

static const int RXPin = 2, TXPin = 3;

static const uint32_t GPSBaud = 9600;

SoftwareSerial gpsSerial(RXPin, TXPin);

TinyGPSPlus gps;

//SIM800L setup

static const int SIM_TX = 10; // Arduino RX

static const int SIM_RX = 11; // Arduino TX

SoftwareSerial sim800(SIM_TX, SIM_RX);

// Contacts

const char *contacts[] = { "+918438471274", "+918958352218" };

const int NUM_CONTACTS = 2;

LiquidCrystal_I2C lcd(0x27, 16, 2);

//Timer

unsigned long accidentStartTime = 0;

bool waitingForConfirmation = false;

void setup() {

  Serial.begin(9600);

  gpsSerial.begin(GPSBaud);

  sim800.begin(9600);

  pinMode(FSR_PIN, INPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  pinMode(CONFIRM_PIN, INPUT_PULLUP);

  pinMode(ALERT_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, HIGH);

  digitalWrite(ALERT_PIN, LOW);

  Wire.begin();

  Wire.beginTransmission(MPU_ADDR);

  Wire.write(0x6B);

  Wire.write(0);
```

```

Wire.endTransmission(true);

lcd.init();

lcd.backlight();

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("System ready");

Serial.println("System ready...");

// Initialize SIM800L

delay(1000);

sendAT("AT");

sendAT("ATE0");    // Echo off

sendAT("AT+CMGF=1"); // SMS text mode
}

void loop() {

  int fsrVal = analogRead(FSR_PIN);

  Wire.beginTransmission(MPU_ADDR);

  Wire.write(0x3B);

  Wire.endTransmission(false);

  Wire.requestFrom(MPU_ADDR, 6);

  int16_t ax = (Wire.read() << 8) | Wire.read();

  int16_t ay = (Wire.read() << 8) | Wire.read();

  int16_t az = (Wire.read() << 8) | Wire.read();

  float ax_g = ax / 16384.0;

  float ay_g = ay / 16384.0;

  float az_g = az / 16384.0;

  float accMag = sqrt(ax_g * ax_g + ay_g * ay_g + az_g * az_g);

  // Condition

  if (!waitingForConfirmation && (fsrVal > FSR_THRESHOLD || accMag > ACC_THRESHOLD)) {

    Serial.println("Potential accident detected! Press button to cancel within 30s.");

    digitalWrite(BUZZER_PIN, LOW);

    accidentStartTime = millis();

    waitingForConfirmation = true;
  }
}

```

```

    lcd.clear();

    lcd.setCursor(0, 0);

    lcd.print("ALARM FOR");

    lcd.setCursor(0, 1);

    lcd.print("ACCIDENT");

}

//Confirmation

if (waitingForConfirmation) {

    if (digitalRead(CONFIRM_PIN) == LOW) {

        Serial.println("Accident cancelled by user.");

        digitalWrite(BUZZER_PIN, HIGH);

        waitingForConfirmation = false;

        lcd.clear();

        lcd.setCursor(0, 0);

        lcd.print("FALSE ALARM");

    }

    else if ((millis() - accidentStartTime >= 30000) && (digitalRead(CONFIRM_PIN) == HIGH)) {

        Serial.println("Accident confirmed!");

        digitalWrite(BUZZER_PIN, HIGH);

        digitalWrite(ALERT_PIN, HIGH); // send to raspberry pi

        lcd.clear();

        lcd.setCursor(0, 0);

        lcd.print("ACCIDENT");

        lcd.setCursor(0, 1);

        lcd.print("CONFIRMED");

        // Update GPS data

        while (gpsSerial.available() > 0) gps.encode(gpsSerial.read());

        // Build SMS message

        String sms = "Accident detected! ";

        if (gps.location.isValid()) {

            sms += "Location: https://maps.google.com/?q=";

            sms += String(gps.location.lat(), 6);

```

```

    sms += ",";

    sms += String(gps.location.lng(), 6);
  } else {
    sms += "GPS not fixed.";
  }

  // Send SMS to all contacts
  for (int i = 0; i < NUM_CONTACTS; i++) {
    sendSMS(contacts[i], sms);
  }

  waitingForConfirmation = false; // Reset after sending }
}

delay(200); }

// --- Functions ---

void sendAT(const char *cmd) {
  sim800.println(cmd);
  delay(500);
  while (sim800.available()) {
    Serial.write(sim800.read());
  }
}

void sendSMS(const char *number, String message) {
  Serial.print("Sending SMS to ");
  Serial.println(number)
  sim800.print("AT+CMGS=\"");
  sim800.print(number);
  sim800.println("\"");
  delay(500);
  sim800.print(message);
  delay(200);
  sim800.write(26); // CTRL+Z to send
  delay(5000);
  Serial.println("SMS sent.");}

```

Code on Raspberry Pi:

```
import cv2

import numpy as np

from picamera2 import Picamera2

import mediapipe as mp

import tfLite_runtime.interpreter as tflite

from twilio.rest import Client

# Twilio credentials

ACCOUNT_SID = "AC890846703d984ff593a5bf30891617d4"

AUTH_TOKEN = "862f3ae61a242e295ed3d57befac48c1"

FROM_PHONE = "+17177224837" # Twilio phone number

TO_PHONE = "+918958352218"


# Initialize Twilio client

client = Client(ACCOUNT_SID, AUTH_TOKEN)

# Initialize camera and models

picam2 = Picamera2()

picam2.start()

mp_hands = mp.solutions.hands

hands = mp_hands.Hands(static_image_mode=True, max_num_hands=1)

interpreter = tflite.Interpreter(model_path="model.tflite")

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()

# Capture and process frame

frame = picam2.capture_array()

image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

results = hands.process(image_rgb)

# Hand detection logic

if results.multi_hand_landmarks:

    print("Hand detected!")
```

```

# Save image

filename = "hand_capture.jpg"

cv2.imwrite(filename, frame)

print(f"Image saved as {filename}")


# Prepare landmarks for TFLite model

hand_landmarks = results.multi_hand_landmarks[0]

landmark_list = []

for lm in hand_landmarks.landmark:

    landmark_list.extend([lm.x, lm.y, lm.z])

input_data = np.array(landmark_list, dtype=np.float32).reshape(1, -1)


# Run TFLite model

interpreter.set_tensor(input_details[0]['index'], input_data)

interpreter.invoke()

output_data = interpreter.get_tensor(output_details[0]['index'])

predicted_class = np.argmax(output_data)

print(f"Predicted gesture class index: {predicted_class}")

# Send SMS alert

message_body = f" Alert: Hand gesture detected! (Class {predicted_class})"

try:

    message = client.messages.create(

        body=message_body,

        from_=FROM_PHONE,

        to=TO_PHONE

    )

    print(f" SMS sent successfully! SID: {message.sid}")

except Exception as e:

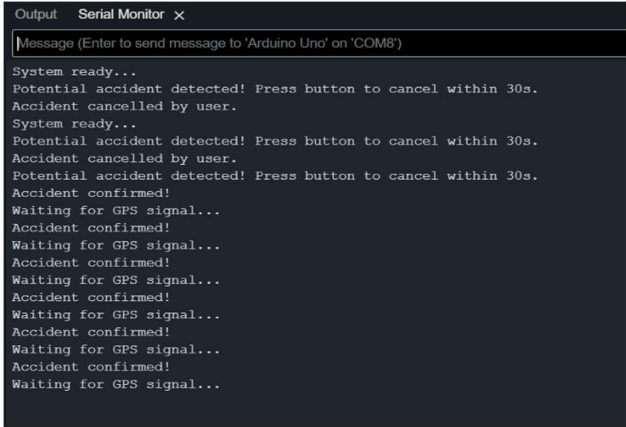
    print(f"Failed to send SMS: {e}")

else:

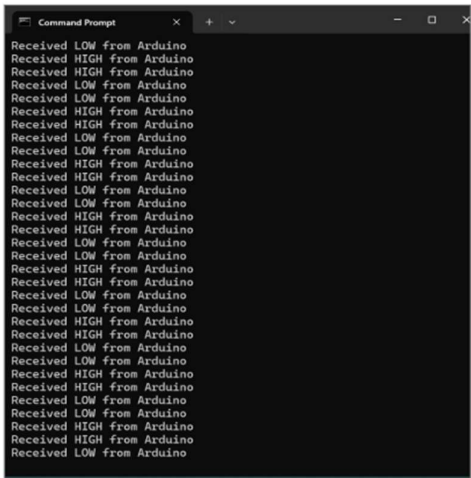
    print("No hand detected")

picam2.stop()

```

```
19:22:34.160 -> Longitude: 80.158164
19:22:34.267 -> Latitude: 12.842939
19:22:34.267 -> Longitude: 80.158164
19:22:35.141 -> Latitude: 12.842938
19:22:35.177 -> Longitude: 80.158164
19:22:35.247 -> Latitude: 12.842938
19:22:35.288 -> Longitude: 80.158164
```



```

51 pinMode(PiSignal, INPUT);
52 }
53
54 // ----- Pi Signal -----
55 // Toggle HIGH/LOW and print status
56 digitalWrite(piSignal, HIGH);
57 Serial.println("Pi Signal: HIGH");
58 delay(1000);
59 digitalWrite(piSignal, LOW);
60 Serial.println("Pi Signal: LOW");
61 delay(1000);
62 }
63

```

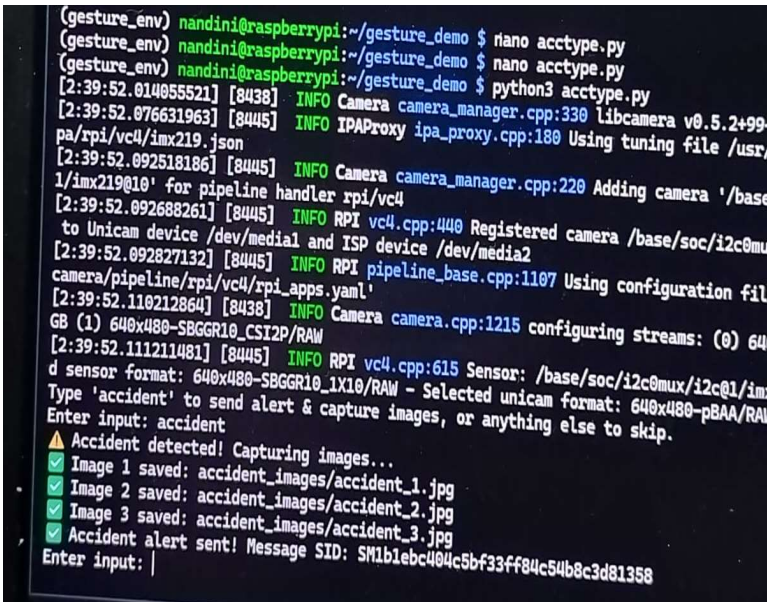
Output **Serial Monitor** ✕

Message (Enter to send message to 'Arduino Uno' on 'COM10')

```

-- Signal: LOW
Pi Signal: HIGH
Pi Signal: LOW
Pi Signal: HIGH
Pi Signal: LOW
Pi Signal: HIGH
Pi Signal: LOW
Pi Signal: HIGH

```



Your Twilio verification code is: 458892

Sent from your Twilio trial account - Hello from your Raspberry Pi! Twilio is working!

06:03

Sent from your Twilio trial account - ⚠ Accident detected! Immediate attention required.

06:03 • SIM1

3.2 HARDWARE (Schematics, PCB Design and Hardware Output)

3.2.1 HARDWARE IMPLEMENTATION ON BREAD BOARD

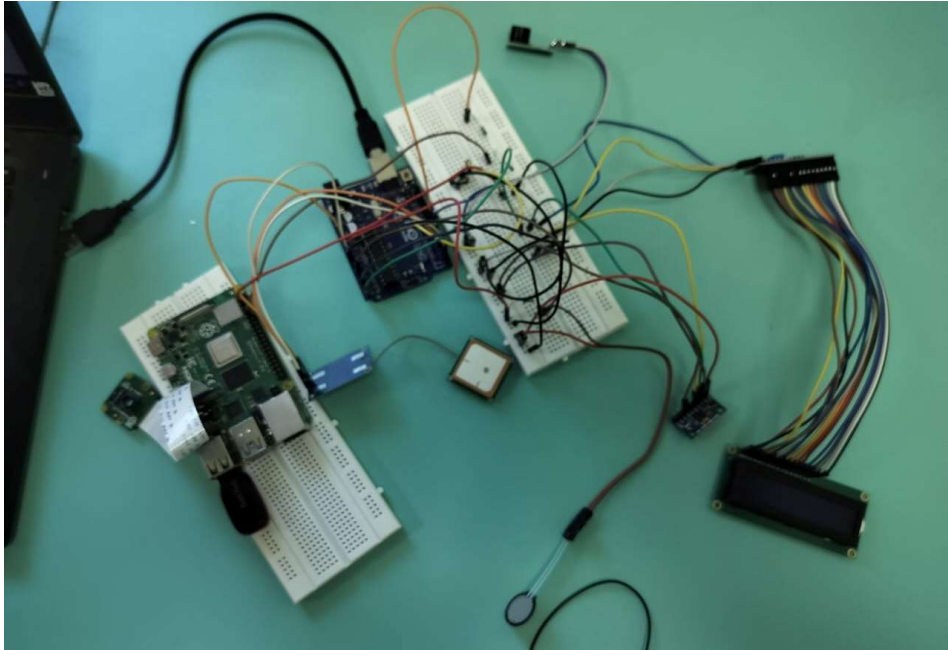


Figure 7: Whole Circuit together (LHS – Pi module, Middle – Arduino Uno, Right – LCD)

Initially, we used a GSM module connected to a Raspberry Pi for SMS functionality, but after it failed, we switched to Twilio for reliable message delivery and easier programmatic control.

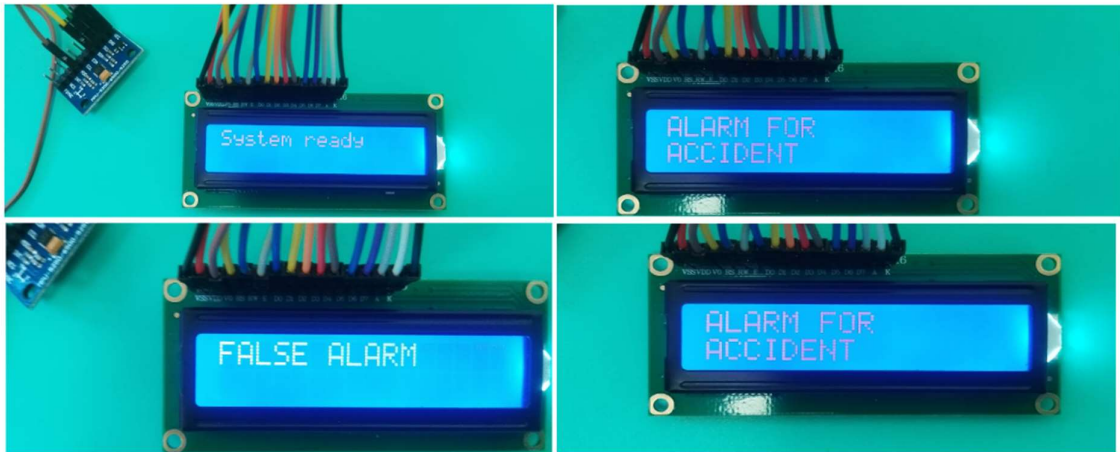


Figure 8: Output Displayed on LCD (Go to references to see video demo)

3.2.2 PCB DESIGN PROCESS

Figure 10: Phase 1 IoT Schematic in KiCad (Labels are used to understand the connections)

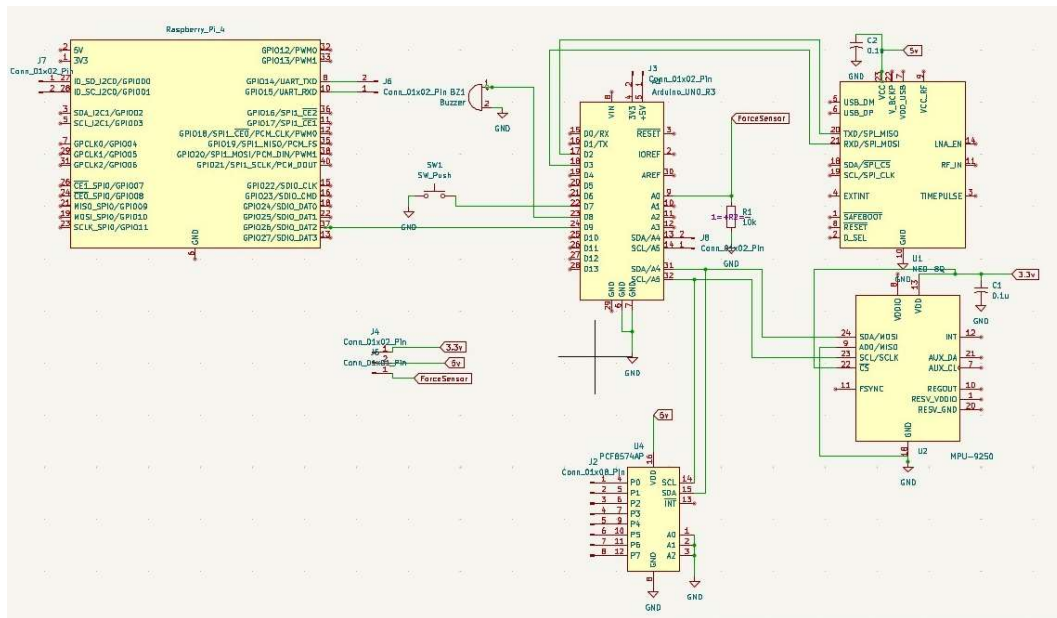


Figure 11: Phase 2 IoT + Raspberry Pi in the Schematic

We decided to only include IoT part for PCB as Raspberry Pi module comes already in a board.

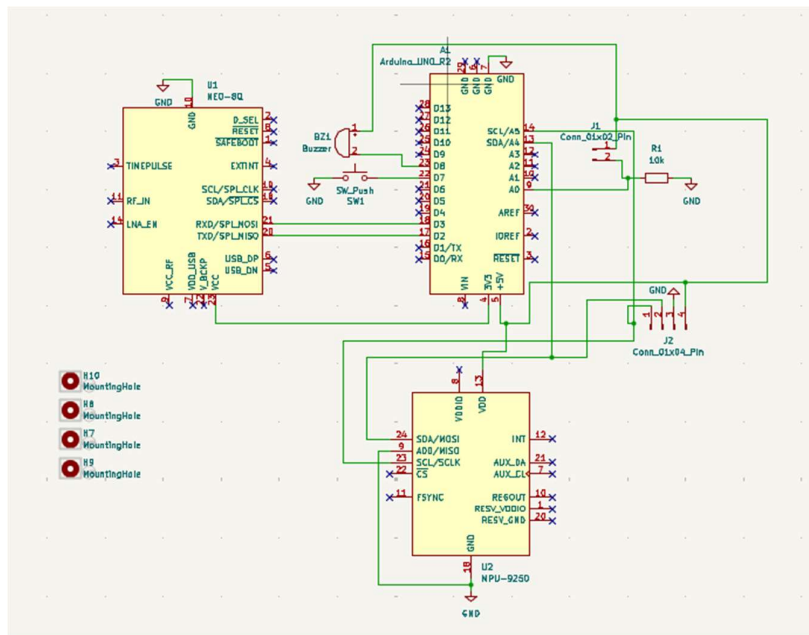


Figure 12: Schematic Designed for PCB design

Again, default footprints available in KiCad 9.0.4 version were used. One can custom design footprints and symbols for sensors used to as actual hardware. We have used 01x02 and 01x04 connector header pins to resemble force sensor and LCD – I2C chip.

Symbol : Footprint Assignments		
A1 -	Arduino_UNO_R2 :	Module:Arduino_UNO_R2
B21 -	Buzzer :	Buzzer_Beeper:Buzzer_15x7.5RM7.6
H7 -	MountingHole :	MountingHole:MountingHole_2.1mm
H8 -	MountingHole :	MountingHole:MountingHole_2.1mm
H9 -	MountingHole :	MountingHole:MountingHole_2.1mm
H10 -	MountingHole :	MountingHole:MountingHole_2.1mm
J1 -	Conn_01x02_Pin :	Connector_PinHeader_2.54mm:PinHeader_1x02_P2.54mm_Vertical
J2 -	Conn_01x04_Pin :	Connector_PinHeader_2.54mm:PinHeader_1x04_P2.54mm_Vertical
R1 -	10k :	Resistor_THT:R_Axial_DIN0204_L3.6mm_D1.6mm_P2.54mm_Vertical
SW1 -	SW_Push :	Button_Switch_THT:SW_PUSH_6mm_H5mm
U1 -	NEO-8Q :	RF_GPS:ublox_NEO
U2 -	MPU-9250 :	Sensor_Motion:InvenSense_QFN-24_3x3mm_P0.4mm

Figure 13: Footprints used

After the placement, routing and copper filling process in PCB design editor was done, DRC rules were checked. At last Gerber file along with drill file was generated.

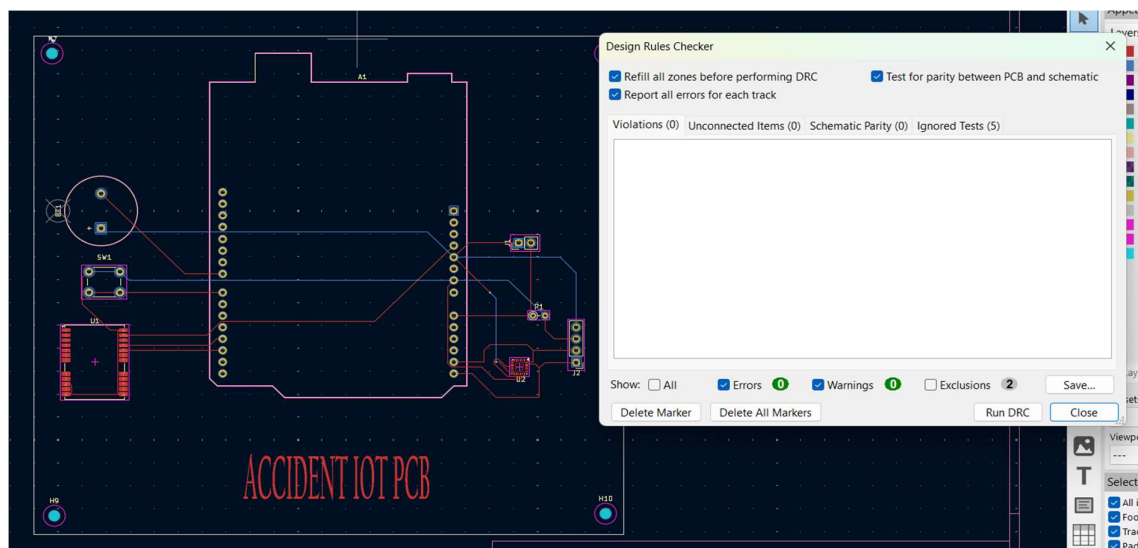
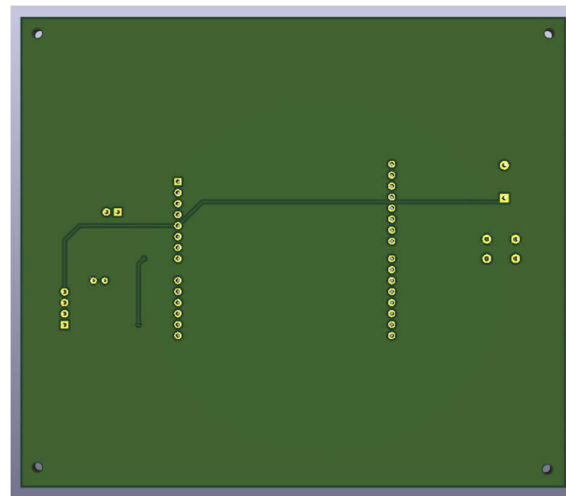
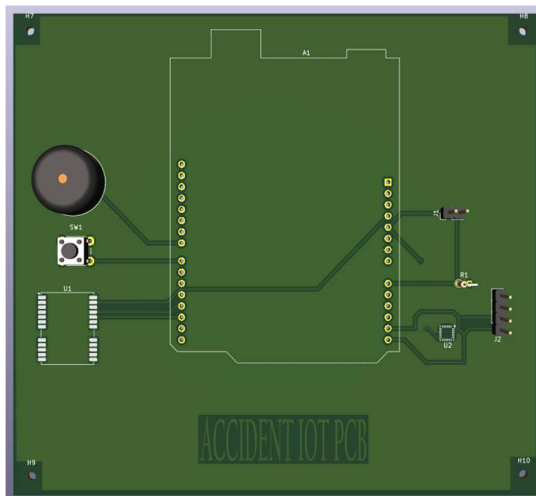
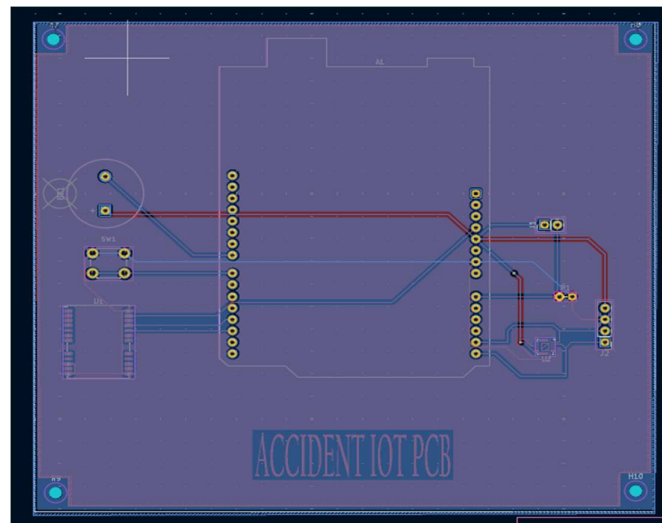


Figure 14: DRC rules check before filling with copper layers



Figures 15: The PCB filled in 3D view; 1st 2D view; 2nd 3D view front; 3rd 3D view back

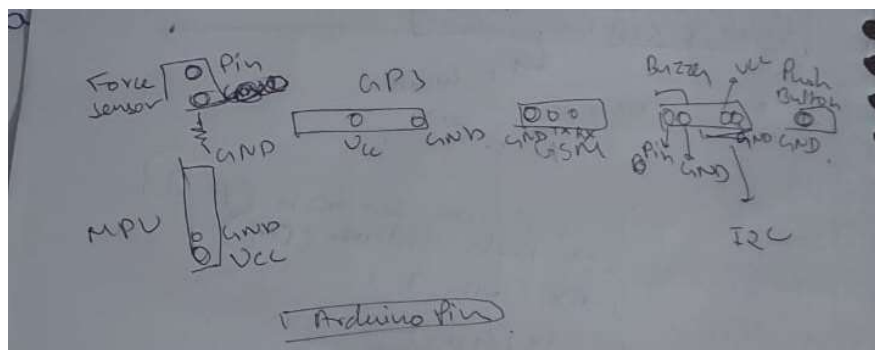


Figure 16: Plan for placing headers on Perf Board

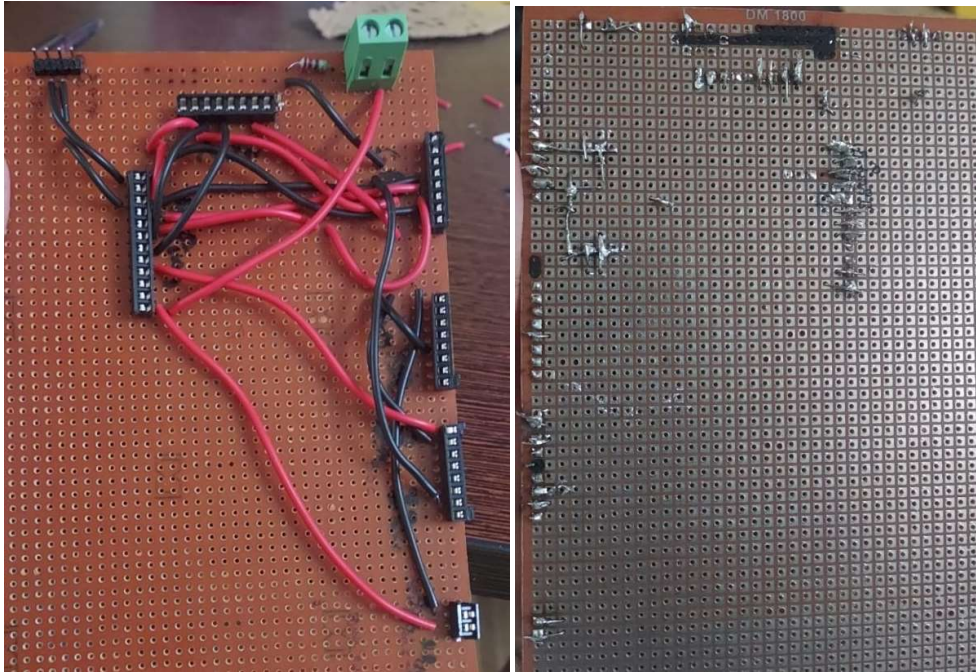


Figure 17: Perf Board Soldering

3.3 RESULTS & DISCUSSION

The proposed AI-enabled accident detection and alert system was successfully designed, implemented, and tested for real-time operation. The IoT module, consisting of Arduino Uno, FSR, and IMU sensors, effectively detected sudden impacts and abnormal acceleration patterns, triggering alerts with high accuracy. The false alarm cancellation mechanism worked reliably within the 30-second response window, reducing unwanted notifications. The GPS and GSM modules accurately transmitted location data, while the Raspberry Pi module successfully received accident signals, captured real-time images, and executed cloud-based severity analysis using the deep learning model. The integration of Twilio for message delivery ensured consistent and fast alert transmission to emergency contacts. The system's end-to-end testing demonstrated low latency, robust sensor performance, and reliable communication between hardware and cloud modules. Overall, the project validates the effectiveness of combining embedded IoT devices with AI for smart accident detection, offering a scalable, cost-efficient, and intelligent solution for enhancing road safety in smart cities.

3.4 COST ANALYSIS (final)

S. No.	Component & Variant	Qty	Unit ₹	Total ₹
1	Arduino Uno	1	550	550
2	Raspberry Pi 4B	1	2,500	2,500
3	Raspberry Pi Camera v2 (8MP)	1	300	300
4	FSR Impact Sensor, IMU Sensor	1	600	600
5	NEO-6M GPS Module	1	500	500
6	SIM800L GSM Module	1	350	350
8	16×2 I2C LCD Display	1	250	250
9	Buzzer + Push Button	1 set	50	50
12	2-layer PCB Prototype	1	800	800
13	Misc. (wires, connectors, enclosure)	-	200	200
	Total			~6000

4. CONCLUSION AND FUTURE WORK

4.1 CONCLUSION

The proposed system successfully demonstrates a real-time, intelligent accident detection and alert mechanism using IoT and deep learning technologies. By integrating Arduino for sensor-based detection and Raspberry Pi for image capture, cloud connectivity, and AI-based severity analysis, the system provides an efficient end-to-end solution for smart transportation safety. The fusion of sensor data (force and IMU) and visual analysis enables reliable accident confirmation while minimizing false alarms. Real-time GPS and GSM modules ensure that accurate location data and emergency alerts reach contacts or authorities without human intervention. The project validates the feasibility of combining embedded systems with artificial intelligence to create a practical, low-cost, and scalable model for road safety in smart city environments.

4.2 FUTURE WORK

Future development of this system can focus on expanding its intelligence, scalability, and autonomy. Enhancements may include implementing cloud-based dashboards for live tracking and data visualization, integrating API connectivity with hospitals or emergency services for automatic dispatch, and training a more robust deep learning model with larger, diverse datasets for improved severity classification. Hardware improvements such as solar-powered operation, miniaturized PCB design, and 4G/5G communication modules can further enhance portability and real-world deployment. Additionally, integrating vehicle-to-infrastructure (V2I) communication will allow coordination between multiple roadside units, paving the way for a fully connected and intelligent accident response network.

5. REFERENCES

Base Paper: <https://www.mdpi.com/2071-1050/14/13/7701>

Code docs:

https://docs.google.com/document/d/1_QhVL9Fj3JYKtcsQ7LwfR1hDtqtOmmPxNZ2mvIcAG6A/edit?usp=sharing

PPT: <https://docs.google.com/presentation/d/147cYPNVAB6wou57vp2-BpIn13-Ye1I51rC-wCiCGUWI/edit?usp=sharing>

Dataset: <https://www.kaggle.com/datasets/asefjamilajwad/car-crash-dataset-ccd>

ML host : <https://nextcloud.com/>

Drive link for KiCad project: https://drive.google.com/file/d/1DXCXeeFBjEFR-C9KaUUcO10ow_mmgoJm/view?usp=sharing

Video Demo: https://drive.google.com/file/d/1VT7v0OX1aPL-mRwqiO36GZwZP_Pez1X1/view?usp=sharing

https://drive.google.com/file/d/18h-i1N0p8UBm_pfbYHjqLJa_WPT5UQ5c/view?usp=sharing